

```

class Solution {
    public int maxSubarraySumCircular(int[] nums) {
        int totalSum = 0;
        int maxSum = nums[0], curMax = 0;
        int minSum = nums[0], curMin = 0;
        for (int num : nums) {
            // Kadane's for max subarray
            curMax = Math.max(num, curMax + num);
            maxSum = Math.max(maxSum, curMax);
            // Kadane's for min subarray
            curMin = Math.min(num, curMin + num);
            minSum = Math.min(minSum, curMin);
            totalSum += num;
        }
        // If all numbers are negative, maxSum is the answer
        if (maxSum < 0) return maxSum;
        // Otherwise, take max of normal and wrap-around case
        return Math.max(maxSum, totalSum - minSum);
    }
}

```

Accepted 112 / 112 testcases passed

Mahipal_Reddy_VTU25525 submitted at Feb 11, 2026 10:29

Editorial

Solution

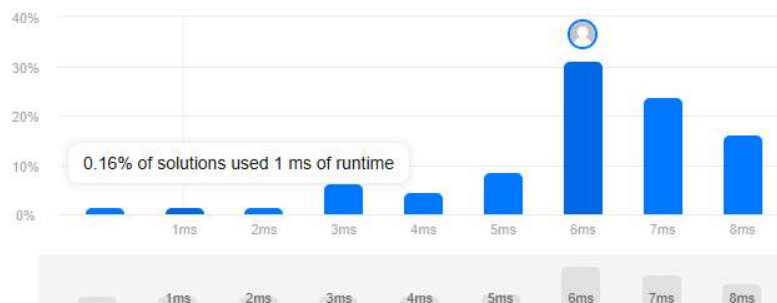
Runtime

6 ms | Beats 78.55%

Analyze Complexity

Memory

51.40 MB | Beats 20.85%



Code | Java

```

1 class Solution {
2     public int maxSubarraySumCircular(int[] nums) {
3         int totalSum = 0;
4         int maxSum = nums[0], curMax = 0;
5         int minSum = nums[0], curMin = 0;
6
7         for (int num : nums) {
8             // Kadane's for max subarray

```

View more